CogLean: Compositional Proof of Cognitive Predictions

Jeff Shrager

Draft for circulation, October 2026

Abstract

Cognitive science has many computational theories, but little cumulative theoretical knowledge. Theories are implemented as separate programs, so combining two of them usually means writing a third, and what a theory predicts is usually established by running it. We describe CogLean, a library of cognitive theory in the Lean proof assistant, organized on the model of Mathlib, the community library of formalized mathematics. CogLean is not a new theory of cognition. Its shared library, CogLib, holds representations, mechanisms, theory-neutral statements of phenomena, and theorems about them. Existing theories are expressed as choices of those mechanisms, and their consequences are proved, not simulated.

The central demonstration composes two independently formalized theories: EPAM's discrimination-net recognition (Feigenbaum and Simon) and the distributions-of-associations model of strategy choice (Siegler and Shrager). They are joined with a problem solver in the GPS and repair-theory tradition to produce a model of three-digit addition. Every component, and every theorem about it, was established before the composition. The composed model computes all of its answers; no route is given the correct sum. Its predictions are proved by reusing those theorems. One is the *confident error*: a problem confused with a familiar one gets the familiar problem's answer, wrongly, with no more use of the backup procedure than the familiar problem itself. Neither theory makes this prediction alone. Others say that improving the fallback solver cannot reduce such errors, and that a confused problem has two sources of error with different signatures. Replacing EPAM's memory with ACT-R's declarative memory, and changing nothing else, changes these predictions, and laws of memory that one memory has and the other lacks explain why.

We separate results that hold for every setting of the mechanisms from results that need stated conditions, and both from exact values at illustrative settings. We also report what an independent, blind simulation found that the proofs did not: hypotheses that learned networks never reach, and an unstated assumption on which a predicted learning sequence depends.

1 Introduction

Cognitive science has accumulated many computational theories of memory, learning, problem solving and skill. It has accumulated much less theory that can be *reused*. A theory of memory developed for verbal learning usually has to be reimplemented before it can take part in a model of arithmetic. When two theories are combined, the result is a new program, whose behavior must be explored afresh by simulation. Newell's complaint that the field asks nature twenty questions without assembling the answers [1], and his call for unified theories [2], are about this problem. So, more recently, are arguments that computational modeling forces theories to be made explicit [3] and that psychology needs explanatory theories, not only tests [4].

Mathematics faced a version of the same problem and has made progress on it. Mathlib [5], the community library of formalized mathematics for the Lean proof assistant [6], does not propose new mathematics. It provides one shared formal environment, in which definitions are made once,

every result is machine-checked, and later work builds on earlier results by citing them. A theorem about groups, once proved, is available to everyone who uses groups.

This paper asks whether the same organization can work for cognitive theory:

> *If independently developed cognitive theories are represented in a common formal library, can we derive scientifically meaningful consequences from their composition, consequences that would otherwise require building and investigating a new integrated model from scratch?*

We describe CogLean [7], an initial library built to test this, and give a concrete, checked answer in one case. The case is three-digit addition, composed from EPAM's discrimination nets [8, 9], Siegler and Shrager's distributions-of-associations (DOA) model of strategy choice [10], and a problem solver in the tradition of GPS and repair theory [11, 12]. Arithmetic is the demonstration, not the objective. We are not proposing a new theory of children's arithmetic. Multi-digit addition is a familiar task, complex enough to need several mechanisms, and two of the needed theories already exist.

The argument is organized around three capabilities, in increasing order of importance.

1. **Formalization.** Existing theories are stated as definitions, postulates and theorems. Each theorem's assumptions are explicit.

2. **Composition.** Formalized mechanisms are reused in new contexts, and so are the *theorems* about them. A library of reusable cognitive algorithms would be useful software. A library of reusable *results* is a contribution to theory.

3. **Discovery.** Composition establishes consequences that the original theories' authors did not state. Formalization also exposes unstated assumptions, results that depend on parameters, results that hold for whole classes of models, and conditions under which theories diverge.

A caution belongs at the start. Lean checks that each conclusion follows from its premises. It does not check that the premises describe people, nor that a valid derivation is an explanation. Section 7 takes this up directly, and we distinguish throughout what holds generally, what holds under stated conditions, and what was computed at illustrative settings.

2 CogLean

2.1 Organization

CogLean has four layers (Figure 1).

- **Lean**, the formal language and proof checker. CogLean uses only Lean's core, not Mathlib, and every probability is an exact rational number.

- **CogLib**, the general library. It holds:

  - representations: feature codes, finite probability distributions, semantic networks;
  - memory structures: discrimination nets, an ACT-R-style activation memory;
  - processes: discrimination learning, sampled retrieval with a confidence criterion, "recognize, then retrieve, else back up", and a decision cycle with impasses and repairs;

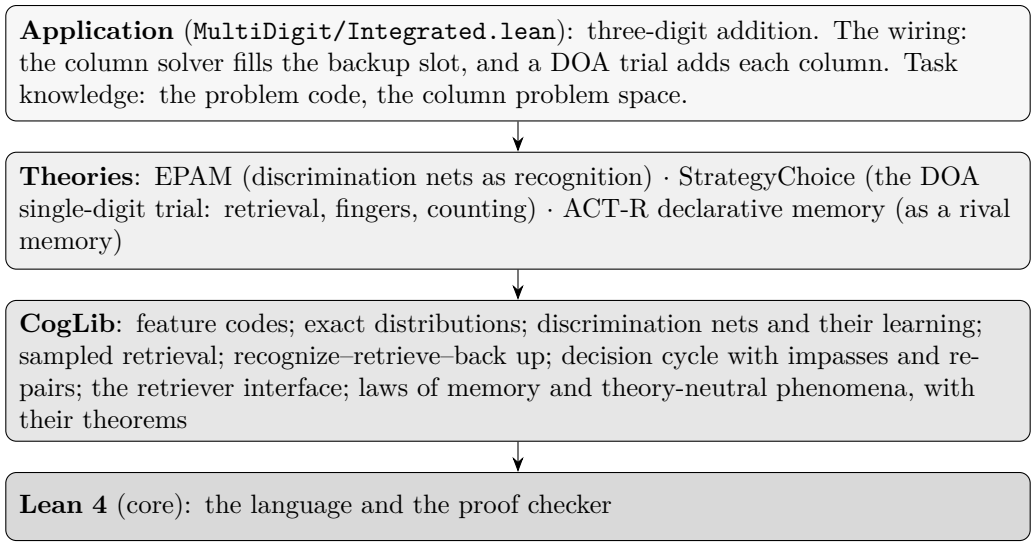


Figure 1: The layers of CogLean. Arrows point from a layer to what it imports. Each layer's theorems are available to the layers above it.

  - an interface any memory can implement (a *retriever*: study an item, retrieve a distribution over stored items);
  - theory-neutral statements of phenomena and laws, such as the serial-position effect, or the law that retrieval of a studied item is certain.

  CogLib imports no theory.

- **Theories**, each a package that states its postulates as choices of CogLib constructs, cited to their sources, and proves its phenomena.
- **Applications**, models of particular tasks assembled from theories and library mechanisms, with the task's own knowledge.

The contribution is not a programming language or a new syntax for cognitive models. Any model could be written in Lean as a program. What CogLean contributes is an *organization* of theoretical knowledge: mechanisms defined once, phenomena stated independently of any mechanism, and theorems relating them, all in one place where later work can cite them. CogLean is far from Mathlib's scale or maturity. It is a demonstration that the approach can work, not a finished library.

2.2 Mechanisms, phenomena, theorems and conditions

Four kinds of object must be kept apart.

1. **Mechanisms and their postulates**: definitions, such as how a discrimination net sorts a code, or how retrieval samples an answer in proportion to its associative strength.
2. **Phenomena**: predicates, stated without reference to any mechanism. A predicate asserts nothing. It is a target, which some mechanism may or may not be proved to satisfy.
3. **Theorems**: proofs that a mechanism produces behavior satisfying a predicate.

4. **Conditions**: the hypotheses under which a theorem holds. Some are situational ("problem x is confused with problem q"), some are empirical premises that remain unexplained ("q's associations are sharp"), and some are well-formedness requirements ("the strengths have positive total").

A theorem that has a phenomenon among its hypotheses is a relation between phenomena, not a prediction of a model. It becomes a prediction only when some mechanism discharges the hypothesis by proving it. CogLean's catalog of theorems, generated from Lean, marks such conditional results mechanically [7].

Shared definitions make theories comparable at the level of proofs. For example, CogLib proves once that any learner whose learning never disturbs what it already knows reaches a learning criterion (`criterion_of_nonInterference`). It separately proves that discrimination-net learning never disturbs prior learning (`netLearner_nonInterference`). EPAM's learning to criterion then follows in a line, and so will that of any later theory whose learning is non-interfering.

3 Two theories, formalized independently

3.1 EPAM: recognition by discrimination

EPAM [8, 9] recognizes a stimulus by sorting its code, a vector of features, down a discrimination net of tests to a stored image. When a stimulus is confused with a stored one, learning grows a test at that point, on the first feature in a noticing order where the two differ. We formalize the propositions of Feigenbaum's report, not the original IPL-V program. CogLib proves, for every net, code and noticing order:

- recognition depends only on the features tested on the path a code follows;
- learning a stimulus never changes the recognition of another (`learn_preserves`);
- a confusion agrees with the stored image on every feature tested on its path (`confusion_agrees_on_path`);
- every net learned from an empty net stores each image once and recognizes each as itself (`learnAll_clean`).

3.2 DOA: distributions of associations

In the DOA model [10], a child adding single digits has associations of varying strength between each problem and possible answers.

1. On each trial the child has a confidence criterion c and a search length n.
2. A search samples an answer in proportion to strength, and states it if its strength exceeds c, for up to n searches.
3. If retrieval is exhausted, the child puts up fingers, which adds strength to the answer that many fingers represents, and retrieves once more.
4. Failing that, the child counts the fingers, with fixed chances of skipping or double-counting an object.
5. Every stated answer is strengthened, a correct one more than an incorrect one.

Let R be the chance that one search clears the criterion. StrategyChoice proves for every distribution, criterion and search length:

- retrieval is used with probability $1 - (1 - R)^n$ (`SC.retrieval_prob`);
- the error rate among retrieval trials is the wrong answers' share of the strength above the criterion, whatever the search length (`SC.retrieval_error_share`);
- neither depends on the backup strategies (`SC.retrieval_independent_of_backup`);
- the error rate among counting trials does not depend on the associations at all, which is the chapter's "nonintuitive prediction" (`SC.counting_error_share`).

Formalization also exposed a gap between the chapter's informal description and its formal consequences. The chapter states that "a peaked distribution results in less frequent use of overt strategies" [10, p. 245]. That is not a consequence of the model's postulates. Moving strength onto the correct answer can push a strong wrong competitor below the criterion, and so *raise* overt strategy use. A setting where this happens, averaged over the chapter's own twelve criteria, is proved in Lean (`SC.peaking_can_raise_overt_use`). The claim needs a condition the chapter does not state.

4 Composing theories: multi-digit addition

4.1 The composition

A child adding two three-digit numbers may know the answer to a familiar problem such as 100+100, or may work it out column by column. The model nests DOA's architecture one level up (Figure 2). It has six parts.

1. **A problem code**: features for each digit of each addend. Agreeing on a digit's features means having the same digit (postulate M1, `MultiDigit/Postulates.lean`).
2. **Recognition** by an EPAM net that stores familiar problems (M2).
3. **Whole-problem retrieval**: DOA sampling from the recognized problem's associations (M3).
4. **A column solver**, invoked when nothing is recognized or retrieval is exhausted (M4). It is a problem space with operators to add a column, write a digit, write and carry, and finish, run by a procedure (`MultiDigit/Solver.lean`).
5. **A DOA single-digit trial** for each column's addition, with retrieval, fingers and counting (`MD.doaFact`, wrapping `SC.trial`).
6. **CogLib's decision cycle**, which sequences the operators and handles carrying, impasses and repairs (`CogLib/Process/ProblemSolving.lean`).

In Lean, the composition is three definitions (`MultiDigit/Integrated.lean`):

```
def solverBackup (F : Facts) (p : Problem) : Dist Nat :=
  Solver.answers F.fact Solver.full Solver.lenient p
def itrial E N str as cc n F x := trial E N str as cc n (solverBackup F) x
```

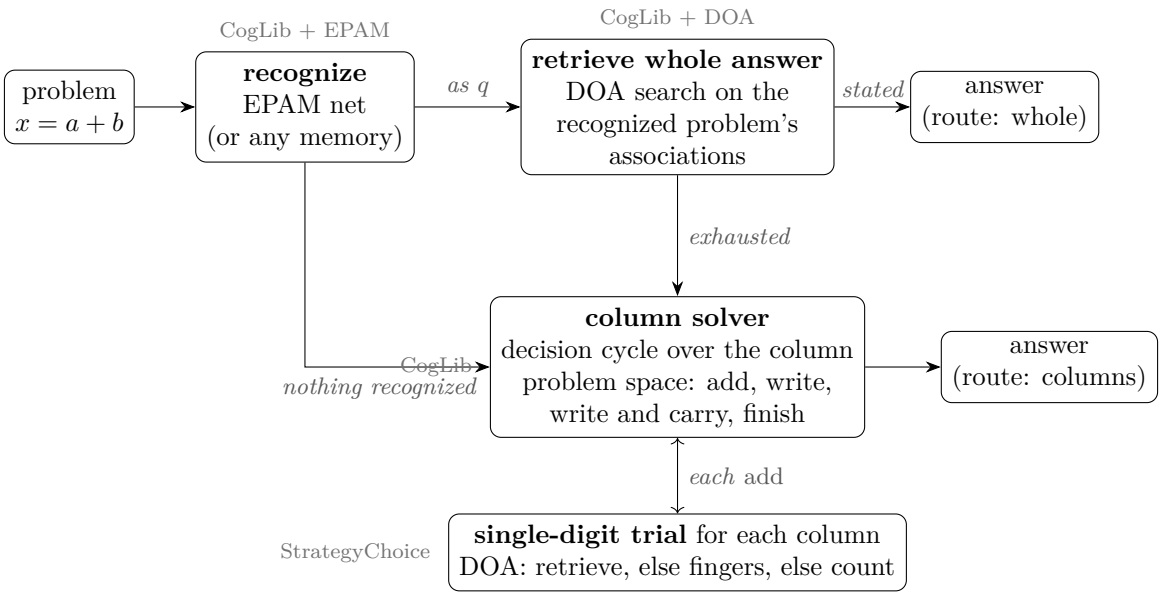


Figure 2: One trial of the integrated model (`MD.Integrated.itrial`). Grey labels give the source of each mechanism. When x is recognized as a familiar q, the whole answer is retrieved from q's associations. The solver runs only when recognition fails or retrieval is exhausted, and it computes the answer from single-digit trials and a carry rule.

Here `F.fact` is the single-digit DOA trial, `Solver.full` the full column procedure, and `trial` the recognize-and-retrieve trial from `MultiDigit/Postulates.lean`. The integrated model introduces no new mechanism. Its only correct arithmetic knowledge is in the single-digit associations, which are peaked on each digit pair's sum, as in DOA, and in the carry rule.

4.2 Why the integrated model is stronger evidence

An earlier version of the example, still in the repository (`MultiDigit/Example.lean`), checked the general theorems with a backup that simply returned the correct sum, `Dist.pure p.sum`. That suffices to instantiate theorems that hold for *every* backup. But it supplies the answer instead of computing it, so it is not evidence that the mechanisms compose. In the integrated model, nothing is handed the answer. The answer 250 to 150+100 is assembled by the solver from three DOA trials and a carry rule, and it is never certain. So the composition must work at the level of mechanisms (the parts fit and run together) and at the level of theorems (the parts' results transfer to the whole).

4.3 Composition of theorems

Three kinds of reuse make the composition deductive rather than merely executable.

A bridge. The solver on CogLib's decision cycle answers exactly as a fixed column procedure does, for every problem and every single-digit process (`Solver.bridge`). Every theorem proved about the fixed procedure therefore holds for the solver. One example is that a column error is a single-digit error moved to its place (`MD.column_error_is_doa_error`).

Associativity. Each arrow in Figure 2 is a two-stage draw of finite distributions. CogLib proves that such draws regroup (`Dist.bind_assoc`), so a theorem about a component holds where the component sits inside the whole.

Well-formedness, inherited. The general confident-errors theorem needs the backup to be a proper distribution, of total probability 1. For the integrated model this is proved, not assumed, by chaining results from three packages (`mass_solverBackup`):

- DOA retrieval is proper (CogLib, `mass_search`);
- a whole DOA trial is proper (StrategyChoice, `SC.mass_trial`);
- the column procedure is proper when its facts are (MultiDigit, `mass_columns`);
- the solver equals the column procedure (`Solver.bridge`).

These lemmas contribute no psychology. They are what allows theorems written for an abstract backup to apply to a concrete, computing one.

5 Derived consequences

We present four results, chosen because their explanatory status differs. The first is explained in detail.

5.1 Confident errors

In DOA alone, a problem's errors and its use of backup strategies come from the same distribution. A flat distribution of associations produces both more retrieval errors and more backup use, so errors and hesitation go together. Recognition breaks this tie. A problem confused with a familiar one is retrieved from the familiar problem's distribution: with that problem's low backup use, and with all of its answers, which are wrong for the confused problem.

The theorem, as stated for the integrated model (lightly abridged):

```
theorem integrated_confident_errors (E : Encoding) (N : Net) (str ...) (F : Facts)
    (hF : F.Proper) (x q : Problem) (hT : ∀ y, total as (str y) ≠ 0)
    (hq : N.select (E.code q) = some (E.code q))      -- q is recognized as itself
    (hx : N.select (E.code x) = some (E.code q))      -- x is confused with q
    (hs : Sharp str cc (E.code q) q)                  -- only q's sum clears the criterion
    (hne : x.sum ≠ q.sum) :
    prob (itrial ... q) (wholeWrong q) = 0 ∧                            -- (i)
    prob (itrial ... x) (wholeWrong x) = prob (itrial ... x) isWhole ∧  -- (ii)
    prob (itrial ... x) isColumns = prob (itrial ... q) isColumns     -- (iii)
```

In words: (i) whole-problem retrieval never errs on q; (ii) every whole-problem answer to x is wrong; (iii) x goes to the column solver exactly as often as q does. The confused problem is answered wrongly with the same fluency as the familiar one.

What is assumed. The postulates are the mechanisms: the net, DOA retrieval and the solver. The conditions are of three kinds.

- *Recognition conditions* (`hq`, `hx`): q is recognized as itself and x is confused with it. These are properties of a particular net. They are not arbitrary: every net EPAM learns recognizes its images as themselves (`learnAll_clean`), and the net in our setting, learned by EPAM from $100+100$ and $100+200$, is verified to confuse $150+100$ with $100+100$ (`Example.net_facts`).
- *Sharpness* (`hs`): only q's correct sum clears the criterion. This is an empirical premise that the library does not yet derive from learning.
- *Well-formedness* (`hF`, `hT`): strengths have positive totals.

No hypothesis mentions confident errors, or any relation between errors and backup use.

What is reused. The proof is one line. It applies `MD.confident_errors`, the theorem for an arbitrary backup, with `mass_solverBackup` for its well-formedness hypotheses. `MD.confident_errors` is in turn a relabelling of CogLib's `RR.confident_errors`, which holds for any task with problems, a familiar set and a backup. That theorem combines three earlier lemmas:

- a problem recognized as a sharp q gets only q's answer (`confusion_gives_neighbours_answer`);
- so all its whole-problem answers are wrong when the sums differ (`wrong_eq_whole_of_sharp`);
- backup use after certain recognition as y is $(1 - R_y)^n$, which depends only on y (`backup_prob_certain`).

The last lemma is DOA's retrieval-probability result, lifted to a recognized item.

Generalizing over memories. The same pattern is proved from laws of memory instead of from a particular net (`RR.confident_errors_of_laws`). Any memory with three properties produces confident errors whenever a stranger can be taken for a studied item with sharp associations:

- *determinacy*: retrieval calls up a single item for certain;
- *certainty*: a studied item retrieves itself;
- *stability*: studying one item does not change another's retrieval of itself.

This is a result about a class of models, not about one model.

5.2 Whole-problem errors do not depend on the solver

Once x is recognized as q, the chance of a whole-problem answer with any property is that of retrieval on q's associations, whatever the backup (`whole_independent_of_solver`). This is a general consequence of the composition, at every setting. Its psychological content is a prediction about intervention: improving column skill, for instance by teaching carrying, cannot reduce errors that arise from mistaken recognition. Only learning that changes recognition or associations can.

5.3 Two sources of error

Under the same conditions as confident errors, the integrated model distinguishes two sources of error on a confused problem (`errors_two_sources`).

- **Whole-route errors** are always the confused neighbour's answer (here 200), whenever that route is taken.
- **Column-route errors** occur with probability equal to the chance that retrieval is exhausted times the solver's own error rate on x. The solver's errors are single-digit DOA errors, displaced to their column: off by $10^j\delta$ when column j errs by δ (`solver_error_is_doa_error`).

The two sources leave different signatures: a whole-route error is another familiar problem's sum, while a column-route error differs from the correct answer in one digit. This result is *compositional*. Neither recognition nor DOA says it alone, and it would not hold with the old backup, which never erred. The model also says when the column route is exact: only when every column's single-digit trial is exact (`solver_exact_of_exact`). DOA sampling does not guarantee that.

5.4 Memory substitution

Because recognition is stated for any retriever, EPAM's net can be replaced by ACT-R's declarative memory [13], with everything else unchanged: the encoding, the associations, the criterion, the search length and the solver. CogLib implements an ACT-R-style memory with base-level activation, partial matching and a retrieval threshold, for decay 1 and temperature 1, so that probabilities stay rational (`CogLib/Memory/Activation.lean`).

The two memories differ in the laws just listed. Both cells are proved, for every parameter setting:

- the discrimination net has determinacy, certainty and stability (`Matrix.Discrimination.determinate`, `.certainty`, `.stability`);
- ACT-R's memory has none of them (`Matrix.Activation.never_determinate`, `never_certainty`, `never_stability`).

So the general confident-errors theorem applies to EPAM and never to ACT-R. Its hypotheses can never be met, because nothing is retrieved for certain (`Act.retrieve_not_certain`). Table 1 shows the consequences at one setting.

With EPAM, the confusion is certain and confident errors dominate. With ACT-R, the confusion is graded. $150+100$ can retrieve either familiar problem, or fail. It therefore reaches the solver far more often and is answered correctly about a quarter of the time. ACT-R also predicts interference: studying $100+300$ roughly doubles the errors on the practiced $100+100$ (`act_interference`). EPAM predicts none, for every net (`MD.learning_preserves_trial`). The difference is between kinds of memory, and the laws say which property of memory produces it.

6 Three kinds of results

Table 2 sorts the results of this section by what they rest on. The distinction matters for credibility. A reader should be able to tell which statements are claims about a whole class of models, which are claims given particular conditions, and which are only numbers at chosen settings.

	EPAM net	ACT-R memory
$150 + 100$ retrieves $100 + 100$	certainly	with probability 10/23
$150 + 100$ answered 200 by whole-problem retrieval	45/46	$225/529 \approx .43$
$150 + 100$ answered 300 (taken for $100 + 200$)	0	$225/1058 \approx .21$
$150 + 100$ reaches the column solver	1/46	$383/1058 \approx .36$
$150 + 100$ answered correctly (250), by the solver	$\approx .014$	$\approx .238$
errors on the practiced $100 + 100$	0	$225/782 \approx .29$
the same, after also studying $100 + 300$	0	$3375/5842 \approx .58$
for every setting (general theorems)	confident errors; practice never disturbed	no certain retrieval; interference possible

Table 1: The same application with two memories. All values are exact, proved in Lean (`ExampleACTR.lean`, `Integrated.lean`), at one illustrative setting: the binary-coded encoding, both memories having studied $100 + 100$ and then $100 + 200$, the same associations, criterion 1/2, search length 1, and the same solver. ACT-R's parameters are a mismatch penalty $\mu = 1/2$ over 24 features and a failure weight $\theta = 1/10$. The last row states what holds at every setting.

The illustrative setting. The encoding codes each digit of each addend in four bits. The net is learned by EPAM from $100 + 100$ and $100 + 200$, scanning hundreds digits first, so it tests only the bit where those two problems differ. The whole-problem associations give strength .90 to each familiar problem's correct sum and .01 to other answers, with criterion .50 and search length 1. Every digit pair has the same single-digit associations: 4/5 on its sum, 3/25 on the next number in the counting sequence, and 1/100 on each other answer from 0 to 19. The single-digit criterion is 1/10, the search length 1, and finger elaboration, skipping and double-counting are each 1/20.

At this setting, $150+100$ is answered 200 by whole-problem retrieval with probability 45/46 and reaches the solver with probability 1/46. The solver alone answers 250 with probability $\approx .656$. Its main errors are 260, 350 and 251, each $\approx .1$: a tens error, a hundreds error (the counting-sequence neighbour of $1 + 1$) and a units error. So $150 + 100$ is answered correctly with probability $\approx .0143$ (`setting_routes`, `setting_solver_x`, `setting_overall_x`). The familiar $100 + 100$ is answered correctly with probability $\approx .993$.

These numbers are exact consequences of the setting, and they show that the conditional theorems' hypotheses can be met by a learned net. They are *not* fitted to children's data and are not predictions about children. Computing them required a small addition to the library. The DOA counting process, written out as a list of outcomes, lists 3^k of them, too many for the kernel to evaluate. A merged evaluator, proved to be the same distribution (`SC.same_trialMerged`, `same_fastBackup`), makes the computation take seconds while every value remains a probability of the model itself.

7 The scientific status of proofs

Validity is not empirical adequacy. A proof establishes that a conclusion follows from its premises. It does not establish that the premises are true of people. Confident errors follow from CogLean's formalization of EPAM and DOA. Whether children make them, and whether they make them for this reason, are empirical questions.

Validity is not explanation either. For a theorem to be an explanation, its premises must characterize mechanisms independently, and not restate the phenomenon. An earlier version of

Kind	Results	What they rest on
General: hold for every encoding, net, strength set and DOA setting	the solver equals the column procedure; whole-problem errors are independent of the solver; the column route factors as exhaustion times the solver; a solver error is a single-digit error moved to its column; the solver is a proper distribution	the mechanisms alone (plus positive strengths, for the last)
Conditional: hold given stated conditions	confident errors; two sources of error; exactness of the solver only given exact facts; confident errors for any memory with three laws	recognition conditions (derivable for EPAM nets, verified for the learned net); sharp associations (an unexplained empirical premise); for the last, the laws themselves
At chosen settings: exact values	150 + 100: 200 by retrieval 45/46; solver reached 1/46; solver correct $\approx .656$; correct overall $\approx .0143$; the ACT-R column of Table 1	an encoding, a learned net, associations, criterion and search length chosen for illustration

Table 2: The integrated model's results, by kind. Lean names are in Appendix A.

CogLib's serial-position result illustrates the difference. Its mechanism was a schedule on which item i was learned on trial twice its distance from the nearer end. Proving that this produces a bowed error curve was valid, but the bow was in the definition. The current version derives the order of learning from a rule over any set of anchors: items nearer an anchor are attended first. The same rule then has a second consequence, the isolation (von Restorff) effect, when a distinctive item serves as an anchor (`CogLib/Control/Serial.lean`). The result is still close to its premises. But one mechanism now answers to two phenomena, which is a real test.

The repository records an explicit audit of this kind for its headline results (`notes/EXPLANATORY_AUDIT.md`). It sorts each result by how far it lies from its premises:

- *definitional*: a restatement of the definitions;
- *close*: the pattern is largely in the mechanism;
- *derived*: it follows from premises that do not mention it;
- *compositional*: it needs two or more mechanisms working together.

Not every CogLean theorem is a psychological explanation, and we do not claim that. Some are properties of algorithms: the soundness of means-ends search, or the merge properties of semantic networks. Those are needed library facts, but they explain nothing on their own. Some are relations between laws. Some, such as confident errors, the two sources of error and solver independence, are compositional: no single mechanism contains them. Making these distinctions visible is part of the method. Lean shows every premise of every theorem; what remains is a scientific judgment, and the right place for it is written down, in the open.

Premises still unexplained. The audit also lists what is assumed but not yet derived:

- sharp associations, in every confident-errors result;
- the prior associations of the worked cases;

- which items serve as anchors in serial learning.

These are the library's most important targets.

8 Checking the formalization

Lean guarantees that the theorems follow from the definitions. It does not guarantee that the definitions say what the prose model says. We tested that link with an independent implementation that could not simply copy the first [cf. N-version programming, 14].

Method. The model was written as a prose specification, with no Lean names, theorems or predictions (`MultiDigit/sim/SPEC.md`). Points the prose leaves open were listed for the implementer to settle and document. Before any simulator existed, a list of checks was committed (`PREREG.md`). Each check named a theorem, the observable statistic, the hypothesis as read from the simulator's logs, and a tolerance: zero violations for exact relations, and frequencies within 4σ for probabilities. A check whose hypothesis never arose was to be reported as vacuous, not as passed. A fresh AI agent, given only the specification and working in an empty directory, wrote the simulator. An audit of its tool calls showed that it read only files it had written. Lean exported exact trial distributions for four instances, and a harness ran the checks on 30 random fixed instances and 20 random learning instances.

Agreement. Every check whose hypothesis arose passed, with no failures. 243 outcome frequencies matched Lean's exact distributions. The predictions about routes, confident errors, carrying, column errors, zeros and thresholds, and learning (non-interference, discrimination, then column work) passed on hundreds to hundreds of thousands of tests each. The simulation predates the problem solver. Because the solver is proved equal to the column procedure the simulation implements (`Solver.bridge`), its results carry over.

What the simulation found that the proofs did not. Three findings remain open in the current library.

1. **Unreachable hypotheses.** Five predictions (near misses within 200 or 20, round problems, and column facts) were vacuous: their hypotheses never held in any net EPAM learned. Each needs a path to test *every* feature of a digit or column. EPAM grows one test per confusion, so paths are short. The theorems are true but say little about the nets the learning mechanism actually produces. Proof alone did not reveal this. Counting how often each hypothesis arose did. The distinction is between logical correctness and *reachable* model states.

2. **A gap at zero strength.** With associations of total strength 0, sampling divides by zero, and a trial has total probability 0. The theorems avoid the case by assuming positive totals. The simulator, reading the prose, treats it as exhausted retrieval, which is the natural reading.

3. **An unstated assumption decides a predicted learning sequence.** The combined model suggests a sequence: a confident error, then, after discrimination, slow column work, then fluent correct retrieval. Whether it occurs depends on the initial associations of a newly discriminated problem, which the prose left open. With none, 40 of 48 discriminated problems followed the sequence. With only the correct answer, the column phase was skipped. With a copy of the neighbour's associations, the error was stated, strengthened and stated again:

7 of 44 problems were trapped, and 35 were still mixed at the end. The blind implementer flagged this dependence from the specification alone.

The simulation does not prove the formalization faithful to the intended theory. It supplies independent evidence that the formal model behaves as its prose describes, and it found where the formal model says too little. Three standards are involved, and they are different: logical correctness (Lean), behavior over reachable states (simulation), and empirical adequacy (data about people), which neither addresses.

9 Discussion

9.1 Relation to previous formal work

CogLean is not the first formalization or mechanical verification of a cognitive theory. Formal specification of cognitive architectures goes back at least to a specification of Soar in Z [15]. It covered the architecture completely but at a low level, and it could not be executed. Cooper, Fox, Farringdon and Shallice argued that better theories would come from languages designed for specifying cognitive models, and gave one, Sceptic, an executable specification language in which they implemented several models, including Soar [16]. Stewart and West rebuilt ACT-R in order to separate its core commitments from implementation detail and to explore alternative components [17]. That anticipates CogLean's interest in interchangeable mechanisms, in an executable setting. Ragni and colleagues proposed a formal framework into which architectures such as ACT-R and Soar can be embedded for comparison [18].

For ACT-R in particular, Gall and Frühwirth gave an abstract operational semantics, a translation to Constraint Handling Rules proved sound and complete with respect to it, and a decidable confluence test [19, 20]. Langenfeld, Westphal and Podelski translated ACT-R models to timed automata for exhaustive defect analysis [21], and proved their timed-automaton semantics correct at the level of architectural components [22]. Macklem and Mili proposed specification languages and algorithms for checking cognitive models against requirements [23]. In the context of the Common Model of Cognition [24], Ganeriwala and colleagues have proposed enabling formal verification of cognitive models [25]. They have also compiled Soar models to discrete-time Markov chains for probabilistic model checking in PRISM [26, 27], which addresses stochastic behavior, as CogLean's exact distributions do. Outside architectures, verbal theories have been formalized as logic programs [28] and as communicating automata [29]. Fisher and colleagues derived critical tests of ACT-R from its core architectural assumptions, as theorems over a space of model classes [30], an aim close to CogLean's separation of what follows from a theory's core.

Model verification and theoretical deduction. Three approaches to establishing what a cognitive model does should be kept apart.

- **Simulation** samples executions to estimate behavioral outcomes.
- **Probabilistic model checking**, as in PRISM, analyzes a specified state-transition model exhaustively, often by numerical methods, to establish properties or compute probabilities over its whole state space. This is formal verification, not simulation. It can also treat parameters symbolically, giving a property or probability as a function of them [31].
- **Deductive theorem proving**, as in CogLean, establishes results for whole classes of models that satisfy stated theoretical assumptions. They are independent of particular parameter settings and of any finite enumeration of states.

The important distinction is therefore not between simulation and formal verification, but between *model verification* and *theoretical deduction*. Model checking primarily verifies properties of a particular formalized system, or of a parameterized family of them. CogLean's principal objects are the theories themselves: reusable representations, mechanisms, assumptions and theorems. Its aim is to establish general consequences of those theories, and to derive further results by composing independently formalized mechanisms. The approaches are complementary, not competitors. A model checker can verify a large, detailed model that would be laborious to reason about deductively. A theorem can state what every such model must do, and why.

What is distinctive in CogLean, relative to this work, is a combination of emphases. CogLean does not introduce formal methods to cognitive science. Few of them are entirely new on their own:

- **Theorems over classes of models**, not properties of one model or one parameterized family. CogLean's theorems quantify over every encoding, net, strength set and backup, and, through laws, over every memory.
- **Phenomena and laws stated theory-neutrally**, so that rival mechanisms can be tested against the same statement.
- **Reusable proofs across independently developed theories.** The previous work formalizes one architecture or model at a time. CogLean's central example composes theorems proved separately for EPAM, DOA and a problem solver into results about their combination.
- **Behavioral predictions derived from composition**, with their premises sorted into postulates, conditions and settings.

Whether any single one of these is novel is a matter for a more complete review. What we claim to be new is their combination: a shared, cumulative library of cognitive theory that supports reusable proofs, composition across theories, and deductive discovery.

9.2 What the approach bought

Composition as deduction. The integrated model's predictions follow from results already proved for its parts, with a small number of new lemmas. The proofs say what each prediction rests on. For example, confident errors need the criterion and search length not to depend on the route. A failed empirical test of confident errors would therefore point at specific premises, not at the model as a whole.

Discovery. Formalization and composition produced results the original authors did not state:

- a dissociation between errors and backup use that neither theory produces alone;
- the independence of recognition errors from solver quality;
- a counterexample to an informal claim of the DOA chapter;
- hypotheses that are true but unreachable by the learning mechanism;
- an unstated assumption on which a predicted learning sequence depends.

The comparison of memories adds conditions under which two theories diverge, explained by laws.

Proof and simulation are complementary. The slogan "simulation explores, proof settles" understates the relation. Simulation also checks what the proofs are *about*: whether their hypotheses arise, and whether the definitions say what the prose says.

9.3 Limitations

- The associative strengths in the worked example are chosen, not derived from experience. Siegler and Shrager's priors came from a counting process. Grounding these in learning histories is open.
- The strongest results depend on sharp associations and on particular recognition confusions. Confusions are derived for learned nets; sharpness is not.
- Single-digit DOA assumes that a representation with the right number of objects is available for elaboration and counting.
- Carrying is error-free in the full procedure, and columns are added independently.
- The framework does not establish psychological plausibility. That remains a matter of evidence and judgment.
- Scalability is a serious question. CogLean uses exact rational distributions over finite outcome spaces. Continuous representations, large state spaces and neural models will need different techniques, and exact evaluation already required a merged evaluator here.
- Three findings of the simulation remain open: unreachable hypotheses, the zero-strength gap, and the initial associations of a newly discriminated problem.

9.4 Future work

Natural next steps:

- derive associative distributions from learning histories, so that sharpness becomes a theorem;
- add feedback-driven learning across whole trials, so that the predicted learning sequence can be proved;
- restate the vacuous predictions in terms of the features a net actually tests;
- generalize the retriever interface, so that more memories can be swapped in;
- compose theories in other domains.

The library already contains early work on the last of these: conceptual combination, semantic memory and view application, on a shared semantic-network representation.

9.5 Conclusion

The question was whether theories represented in a common formal library can be composed to yield meaningful consequences that would otherwise require building a new model from scratch. In one case the answer is yes. EPAM's recognition, DOA's retrieval and strategy choice, and a decision-cycle problem solver, each formalized on its own, compose into a model of three-digit addition with no answer supplied. Its predictions, confident errors, solver independence and two

sources of error, are proved by reusing the components' theorems. Replacing one memory with another changes the predictions in ways that laws of memory explain. The larger ambition is a cumulative theoretical science, in which cognitive constructs and their proved properties are reused across models, tasks and research traditions, as mathematical results are reused in Mathlib. One example does not establish that. It shows that the approach can produce results of the right kind.

Code. All definitions and theorems are in the CogLean repository [7], `https://github.com/jeffshrager/CogLean`. Every theorem is fully proved, with no `sorry`, and depends only on Lean's standard axioms.

References

[1] Allen Newell. You can't play 20 questions with nature and win: Projective comments on the papers of this symposium. In William G. Chase, editor, *Visual Information Processing*, pages 283–308. Academic Press, New York, 1973.

[2] Allen Newell. *Unified Theories of Cognition*. Harvard University Press, Cambridge, MA, 1990.

[3] Olivia Guest and Andrea E. Martin. How computational modeling can force theory building in psychological science. *Perspectives on Psychological Science*, 16(4):789–802, 2021. doi: 10.1177/1745691620970585.

[4] Iris van Rooij and Giosuè Baggio. Theory before the test: How to build high-verisimilitude explanatory theories in psychological science. *Perspectives on Psychological Science*, 16(4): 682–697, 2021. doi: 10.1177/1745691620970604.

[5] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.

[6] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[7] Jeff Shrager. CogLean: A library of cognitive theory in Lean 4. `https://github.com/jeffshrager/CogLean`, 2026.

[8] Edward A. Feigenbaum. An information processing theory of verbal learning. RAND Paper P-1817, RAND Corporation, Santa Monica, CA, 1959.

[9] Edward A. Feigenbaum and Herbert A. Simon. A theory of the serial position effect. *British Journal of Psychology*, 53(3):307–320, 1962. doi: 10.1111/j.2044-8295.1962.tb00836.x.

[10] Robert S. Siegler and Jeff Shrager. Strategy choices in addition and subtraction: How do children know what to do? In Catherine Sophian, editor, *Origins of Cognitive Skills*, pages 229–293. Erlbaum, Hillsdale, NJ, 1984.

[11] Allen Newell and Herbert A. Simon. *Human Problem Solving*. Prentice-Hall, Englewood Cliffs, NJ, 1972.

[12] John Seely Brown and Kurt VanLehn. Repair theory: A generative theory of bugs in procedural skills. *Cognitive Science*, 4(4):379–426, 1980. doi: 10.1207/s15516709cog0404_3.

[13] John R. Anderson. *How Can the Human Mind Occur in the Physical Universe?* Oxford University Press, New York, 2007.

[14] Algirdas Avižienis. The N-version approach to fault-tolerant software. *IEEE Transactions on Software Engineering*, SE-11(12):1491–1501, 1985. doi: 10.1109/TSE.1985.231893.

[15] Brian G. Milnes, Garrett Pelton, Robert Doorenbos, Mike Hucka, John Laird, Paul Rosenbloom, and Allen Newell. A specification of the Soar cognitive architecture in Z. Technical Report CMU-CS-92-169, School of Computer Science, Carnegie Mellon University, Pittsburgh, PA, 1992.

[16] Richard Cooper, John Fox, Jonathan Farringdon, and Tim Shallice. A systematic methodology for cognitive modelling. *Artificial Intelligence*, 85(1–2):3–44, 1996. doi: 10.1016/0004-3702(95)00112-3.

[17] Terrence C. Stewart and Robert L. West. Deconstructing and reconstructing ACT-R: Exploring the architectural space. *Cognitive Systems Research*, 8(3):227–236, 2007. doi: 10.1016/j.cogsys.2007.06.006.

[18] Marco Ragni, Kai Sauerwald, Tanja Bock, Gabriele Kern-Isberner, Paulina Friemann, and Christoph Beierle. Towards a formal foundation of cognitive architectures. In Chuck Kalish, Martina A. Rau, Xiaojin Zhu, and Timothy T. Rogers, editors, *Proceedings of the 40th Annual Conference of the Cognitive Science Society*, Madison, WI, 2018.

[19] Daniel Gall and Thom Frühwirth. An operational semantics for the cognitive architecture ACT-R and its translation to Constraint Handling Rules. *ACM Transactions on Computational Logic*, 19(3):22:1–22:42, 2018. doi: 10.1145/3218818. Preprint arXiv:1702.01606 (2017).

[20] Daniel Gall and Thom Frühwirth. A decidable confluence test for cognitive models in ACT-R, 2017. arXiv:1705.08392.

[21] Vincent Langenfeld, Bernd Westphal, and Andreas Podelski. On formal verification of ACT-R architectures and models. In *Proceedings of the 41st Annual Conference of the Cognitive Science Society*, pages 618–624, 2019.

[22] Vincent Langenfeld, Bernd Westphal, and Andreas Podelski. A formal operational model of ACT-R: Structure and behaviour. In *Proceedings of the 43rd Annual Conference of the Cognitive Science Society*, 2021.

[23] Andrea Macklem and Fatma Mili. Formal verification of cognitive models. In *Proceedings of the Nineteenth International Florida Artificial Intelligence Research Society Conference (FLAIRS 2006)*. AAAI Press, 2006.

[24] John E. Laird, Christian Lebiere, and Paul S. Rosenbloom. A standard model of the mind: Toward a common computational framework across artificial intelligence, cognitive science, neuroscience, and robotics. *AI Magazine*, 38(4):13–26, 2017. doi: 10.1609/aimag.v38i4.2744.

[25] Parth Ganeriwala, Miki Matsumuro, Frank E. Ritter, and Siddhartha Bhattacharyya. Enabling formal verification in a common model of cognition. Working paper, SBP-BRiMS 2025, 2025. `https://sbp-brims.org/2025/papers/working-papers/2025_SBP-BRiMS_paper_18.pdf`.

[26] Parth Ganeriwala, Candice Chambers, James I. Lathrop, Siddhartha Bhattacharyya, Junaid Babar, Stephen B. Gilbert, Isaac Amundson, Michael C. Dorneich, and Mohammed Abdul Hafeez Khan. Formal analysis of stochastic cognitive models: A translation framework from Soar to PRISM. In *NASA Formal Methods*, Lecture Notes in Computer Science, pages 192–213, Cham, 2026. Springer. doi: 10.1007/978-3-032-28079-4_9.

[27] Marta Kwiatkowska, Gethin Norman, and David Parker. PRISM 4.0: Verification of probabilistic real-time systems. In *Computer Aided Verification (CAV 2011)*, volume 6806 of *Lecture Notes in Computer Science*, pages 585–591. Springer, 2011. doi: 10.1007/978-3-642-22110-1_47.

[28] Jean-Christophe Rohner and Håkan Kjellerstrand. Using logic programming for theory representation and scientific inference. *New Ideas in Psychology*, 61:100838, 2021. doi: 10.1016/j.newideapsych.2020.100838.

[29] Alain Finkel, Gaspard Fougea, and Stéphane Le Roux. An automata-based method to formalize psychological theories: The case study of Lazarus and Folkman's stress theory. In *Proceedings of MODELSWARD 2025*, 2025. arXiv:2501.05185.

[30] Christopher R. Fisher, Joseph W. Houpt, Othalia Larue, and Kevin Schmidt. Using systems factorial technology for global model analysis of ACT-R's core architectural assumptions. *Journal of Mathematical Psychology*, 125:102924, 2025. doi: 10.1016/j.jmp.2025.102924.

[31] Ernst Moritz Hahn, Holger Hermanns, and Lijun Zhang. Probabilistic reachability for parametric Markov models. *International Journal on Software Tools for Technology Transfer*, 13 (1):3–19, 2011. doi: 10.1007/s10009-010-0146-x.

A Index of results

Claim	Lean name (file)
Discrimination learning preserves prior recognition	`learn_preserves` (`CogLib/Process/DiscriminationLearning`)
A confusion agrees on every tested feature	`confusion_agrees_on_path` (`CogLib/Memory/DiscriminationNet`)
Learned nets recognize each image as itself	`learnAll_clean` (`CogLib/Process/DiscriminationLearning`)
DOA retrieval use, error share, backup independence	`SC.retrieval_prob`, `retrieval_error_share`, `retrieval_independent_of_backup`
Counting errors independent of associations	`SC.counting_error_share`
Peaking can raise overt strategy use	`SC.peaking_can_raise_overt_use`
Confident errors, any task	`RR.confident_errors` (`CogLib/Process/RecognizeRetrieve`)
Confident errors from laws of memory	`RR.confident_errors_of_laws`
Solver equals the column procedure	`Solver.bridge` (`MultiDigit/Solver`)
A column error is a DOA error in place	`MD.column_error_is_doa_error` (`MultiDigit/Columns`)
Integrated: solver is the column backup; proper	`solverBackup_eq_columns`, `mass_solverBackup` (`MultiDigit/Integrated`)
Integrated: whole-problem errors independent of solver	`whole_independent_of_solver`
Integrated: column route factored	`columnsSuch_prob`
Integrated: confident errors; two sources	`integrated_confident_errors`, `errors_two_sources`
Integrated: solver error is a DOA error; exact only given exact facts	`solver_error_is_doa_error`, `solver_exact_of_exact`
Integrated: values at the setting (EPAM; ACT-R)	`setting_routes`, `setting_solver_x`, `setting_overall_x`, `setting_overall_q1`; `setting_actr_x`
ACT-R: graded confusion; interference	`act_values`, `act_interference` (`MultiDigit/ExampleACTR`)
EPAM: practice never disturbed	`MD.learning_preserves_trial`
Memory laws: EPAM has, ACT-R never has	`Matrix.Discrimination.*`, `Matrix.Activation.never_*` (`CogMatrix`)
Merged evaluation is the same distribution	`SC.same_trialMerged`, `same_fastBackup`